

PONTIFÍCIA UNIVERSIDADE CATÓLICA DO PARANÁ

LUCAS SCHAPIEWSKI MORO

TÍTULO: SEGURANÇA RESIDENCIAL

Curitiba
2025

RESUMO

Este trabalho acompanha a jornada de aprimoramento de um código para ESP32, que começou como um sistema de alerta de movimento enviando um link para uma câmera IP via CallMeBot. As modificações buscaram otimizar o desempenho, refinar a detecção de movimento e explorar diferentes formas de feedback visual. Notavelmente, houve uma transição da URL de câmera IP para um ícone textual embarcado e, subsequentemente, para o envio de um link direcionado a uma imagem específica hospedada online, conforme solicitado. O desenvolvimento também envolveu a criação de um ícone temático de 8x8 pixels. São discutidas as explorações sobre a viabilidade de envio direto de arquivos de imagem e a pesquisa por APIs alternativas, contextualizando as limitações encontradas. O objetivo é registrar a evolução do projeto e as soluções implementadas.

ABSTRACT

This document chronicles the enhancement journey of an ESP32 codebase, which began as a motion alert system sending an IP camera link via CallMeBot. Modifications aimed to optimize performance, refine motion detection, and explore different visual feedback methods. Notably, there was a transition from an IP camera URL to an embedded textual icon, and subsequently, to sending a link to a specific user-provided image hosted online, as requested. Development also involved creating a thematic 8x8 pixel icon. Explorations into the feasibility of direct image file transmission and research for alternative APIs are discussed, contextualizing the limitations encountered. The aim is to record the project's evolution and the implemented solutions.

SUMÁRIO

Sumário

Sumário	9
1 INTRODUÇÃO	11
2 CAPÍTULOS DE DESENVOLVIMENTO	11
2.1 O PONTO DE PARTIDA: UM ALERTA SIMPLES COM CÂMERA IP	11
2.2 GANHANDO AGILIDADE: OTIMIZAÇÕES DE VELOCIDADE E RESPOSTA	12
2.3 DETECÇÃO MAIS INTELIGENTE: UM FILTRO PARA O SENSOR PIR	12
2.4 EXPLORANDO MÍDIA EMBARCADA: O ÍCONE TEXTUAL DE 8X8 PIXELS	13
2.4.1 Implementação do Ícone Genérico	13
2.4.2 Personalização: O Ícone "Invasor"	13
2.5 FOCO NA IMAGEM ESPECÍFICA: USO DE URL PARA IMAGEM HOSPEDADA	13
2.6 O DESAFIO DO ENVIO DIRETO DA "IMAGEM EM SI"	13
2.7 O CODIGO ATUAL	14
3. Relatório de Desempenho e Testes do Sistema de Alerta de Movimento	18
3.1 Análise de Latência	18
3.2. Análise de Consumo de Energia	19
3.3 Análise da Eficiência da Comunicação	21
4 Conclusão	22
REFERÊNCIAS	23

1 INTRODUÇÃO

Este documento é o relato de uma série de transformações aplicadas a um código para ESP32. A versão inicial do projeto consistia em um sistema de alerta simples: um sensor de presença (PIR) detectava movimento, LEDs sinalizavam localmente, e uma mensagem era enviada para o WhatsApp através da API CallMeBot. Crucialmente, essa mensagem continha um link para o snapshot de uma câmera IP, permitindo uma verificação visual remota do ambiente.

Contudo, o desenvolvimento é um processo vivo. Impulsionados pelo desejo de aprimorar e adaptar, exploramos diversas modificações. Buscamos maior rapidez e eficiência, uma detecção de movimento mais confiável e, significativamente, diferentes maneiras de apresentar a informação visual no alerta – desde a configuração para usar um celular como câmera IP, passando pela criação de ícones textuais, até a tentativa de integrar uma imagem específica fornecida pelo usuário através de um link para um site de hospedagem..

.

2 CAPÍTULOS DE DESENVOLVIMENTO

Será detalhado as etapas que levaram o código do seu estado inicial até as versões mais elaboradas, explicando as motivações e as implementações.

2.1 O PONTO DE PARTIDA: UM ALERTA SIMPLES COM CÂMERA IP

- Um ESP32 lia um sensor PIR.
- LEDs indicavam a detecção. 3 LEDs (vermelho, amarelo e verde)
- O dispositivo se conectava ao Wi-Fi.
- Utilizava a biblioteca HTTPClient.h para se comunicar com a API CallMeBot e enviar uma mensagem para o WhatsApp.

- A mensagem incluía um texto de alerta e a variável `fullCameraSnapshotUrl`, que era construída a partir de `cameraIpAddress` e `cameraSnapshotUrlPath`, apontando para uma câmera na rede.
- Havia um `delay(30000)`; para evitar mensagens consecutivas.

Este era um sistema funcional, mas com potencial para se tornar mais ágil e adaptável. A configuração da câmera IP, por exemplo, era um ponto que poderia ser flexibilizado (o que foi brevemente explorado com a menção ao app IP Webcam como uma forma de obter um `cameraIpAddress` e `cameraSnapshotUrlPath` de um celular).

2.2 GANHANDO AGILIDADE: OTIMIZAÇÕES DE VELOCIDADE E RESPOSTA

A primeira grande leva de mudanças focou em performance:

- O `delay(30000)`; foi substituído por uma lógica de "cooldown" não bloqueante usando `millis()`, permitindo que o ESP32 continuasse ativo e responsivo.
- Os tempos de acendimento dos LEDs foram reduzidos.
- O `delay()` no final do loop principal também foi diminuído, aumentando a frequência de checagem do sensor. Isso resultou em um sistema que reagia mais rapidamente aos eventos.

2.3 DETECÇÃO MAIS INTELIGENTE: UM FILTRO PARA O SENSOR PIR

Para evitar disparos por sinais falsos ou muito breves do sensor PIR, a lógica de detecção foi aprimorada:

- Foi introduzido um limiar de estabilidade (`MOTION_STABILITY_THRESHOLD_MS`): o sensor precisaria ficar em estado HIGH por um tempo mínimo (ex: 200ms) para que o movimento fosse considerado "confirmado".
Variáveis de estado (`isPotentialMotion`, `potentialMotionStartTime`, `ongoingValidatedMotion`) foram usadas para gerenciar esse processo de validação antes de considerar o disparo de um alerta.

Com isso, a confiabilidade dos alertas aumentou.

2.4 EXPLORANDO MÍDIA EMBARCADA: O ÍCONE TEXTUAL DE 8X8 PIXELS

Em um determinado momento, surgiu o desejo de ter um elemento visual diretamente na mensagem, em vez de depender de um link para uma câmera externa.

2.4.1 Implementação do Ícone Genérico

- As variáveis e a lógica relacionadas à URL da câmera IP foram temporariamente removidas ou comentadas.
- Foi criado um array de bytes (const byte simplelcon[8]) para representar um ícone monocromático de 8x8 pixels.
- Uma função bytesToTextImage() foi desenvolvida para converter esse array em uma String formatada com caracteres (ex: '#' e '.'), que era então incluída na mensagem do CallMeBot.

2.4.2 Personalização: O Ícone "Invasor"

A ideia do ícone textual evoluiu para algo mais temático. O conteúdo do array simplelcon[8] foi modificado para formar um novo padrão de pixels que simbolizasse um "bandido invadindo uma casa", mantendo o formato 8x8 e a mesma função de conversão para texto. Este foi o último estágio da "mídia embarcada" como texto.

2.5 FOCO NA IMAGEM ESPECÍFICA: USO DE URL PARA IMAGEM HOSPEDADA

Após a fase dos ícones textuais, você apresentou uma imagem JPG específica (o bandido abrindo a maçaneta) e solicitou que *essa imagem* fosse usada no alerta. Como o envio direto do arquivo não era viável pela API CallMeBot de texto, a solução implementada no código foi:

- **Remoção do Ícone Textual:** O array simplelcon e a função bytesToTextImage() foram removidos.
- **Introdução de alertImageUrl:** Uma nova variável const char* alertImageUrl foi adicionada. A ideia era que você hospedasse a sua imagem JPG em um serviço online (Imgur, etc.) e colocasse o link direto para ela nessa variável.
- **Atualização da Mensagem de Alerta:** A mensagem do CallMeBot foi modificada para incluir o texto "Imagem do evento:" seguido do conteúdo de alertImageUrl.
O código foi, portanto, adaptado para enviar um link para uma imagem estática específica fornecida por você e hospedada externamente, substituindo tanto o link da câmera IP ao vivo quanto o ícone textual.

2.6 O DESAFIO DO ENVIO DIRETO DA "IMAGEM EM SI"

Mesmo com a solução do link para a imagem hospedada, não era possível enviar a imagem em si por causa que a api do callmebot restringia apenas em mensagem e não era possível enviar imagem.

- A api whatsapp.php do CallMeBot é para texto e não suporta o upload de arquivos de imagem pelo ESP32.
- Para o ESP32 "enviar a imagem", ele precisaria primeiro ter o arquivo JPG armazenado em sua memória interna (SPIFFS/LittleFS), o que é um passo adicional de preparação.
- Para enviar um arquivo de imagem requereria um tipo de API e uma requisição HTTP completamente diferentes (normalmente POST com multipart/form-data).

Conclusão desta fase: Não houve alteração no código para envio direto do arquivo, pois a API em uso não o permitia.

2.7 O CODIGO ATUAL

```
#include <WiFi.h>
#include <HTTPClient.h>
#include <UrlEncode.h>

// --- Configurações de Rede e API ---
const char* ssid = "Redmi Note 7";
const char* password = "jtct7903"; // Considere armazenar senhas de forma
mais segura se necessário

// --- Pinos ---
const int sensorPin = 21; // Pino do sensor de presença
const int ledVermelho = 19; // Pino do LED vermelho
const int ledAmarelo = 18; // Pino do LED amarelo
const int ledVerde = 5; // Pino do LED verde

// --- Configurações da API CallMeBot ---
String phoneNumber = "+554192270628"; // Seu número de telefone com
código do país
String apiKey = "4467246"; // Sua API Key do CallMeBot

// --- URL da Imagem do Alerta ---
// !!! IMPORTANTE: Substitua pela URL DIRETA da imagem que você
hospedou online !!!
// Exemplo: "https://i.imgur.com/seu_codigo_de_imagem.jpg"
const char* alertImageUrl = "https://i.postimg.cc/6QRWKL0K/fa-a-o-codigo-de-
uma-imagem-de-um-bandido-comun-abrindo-uma-ma-aneta-de-uma-porta.jpg";

// --- Definição do tipo de Callback ---
typedef void (*MessageSentCallback)(bool success, int httpCode);

// --- Otimização de Memória: Histórico de Alertas ---
const int MAX_HISTORY_EVENTS = 10;
struct AlertEvent {
    unsigned long timestamp;
    bool messageSentSuccessfully;
    int responseCode;
```

```

};
AlertEvent alertHistory[MAX_HISTORY_EVENTS];
int currentEventIndex = 0;
int eventCount = 0;

// --- Variáveis para controle de tempo do alerta (envio de mensagem) ---
unsigned long lastAlertTime = 0;
const unsigned long alertCooldown = 5000; // Cooldown em ms para ENVIO DE
MENSAGENS.

// --- Variáveis para otimização da detecção de movimento ---
const unsigned long MOTION_STABILITY_THRESHOLD_MS = 200; //
Milissegundos que o sensor deve ficar HIGH para confirmar movimento.
unsigned long potentialMotionStartTime = 0;
bool isPotentialMotion = false;
bool ongoingValidatedMotion = false;

void addAlertToHistory(bool success, int httpCode) {
    alertHistory[currentEventIndex].timestamp = millis();
    alertHistory[currentEventIndex].messageSentSuccessfully = success;
    alertHistory[currentEventIndex].responseCode = httpCode;

    currentEventIndex = (currentEventIndex + 1) % MAX_HISTORY_EVENTS;
    if (eventCount < MAX_HISTORY_EVENTS) {
        eventCount++;
    }
    Serial.println("Evento de alerta adicionado ao histórico.");
}

void sendMessage(String message, MessageSentCallback callback) {
    String encodedMessage = urlEncode(message);
    String url;
    url.reserve(250 + phoneNumber.length() + apiKey.length() +
encodedMessage.length()); // Ajuste a reserva conforme o tamanho da URL da
imagem

    url = "https://api.callmebot.com/whatsapp.php?phone=";
    url += phoneNumber;
    url += "&apikey=";
    url += apiKey;
    url += "&text=";
    url += encodedMessage;

    HTTPClient http;
    Serial.print("URL CallMeBot: ");
    Serial.println(url);

    http.begin(url);
    http.addHeader("Content-Type", "application/x-www-form-urlencoded");
    int httpResponseCode = http.GET();

```

```

bool success = false;
if (httpResponseCode > 0) {
    Serial.print("HTTP response code: ");
    Serial.println(httpResponseCode);
    if (httpResponseCode == 200) {
        Serial.println("Mensagem enviada com sucesso via CallMeBot.");
        success = true;
    } else {
        Serial.println("Erro no envio da mensagem via CallMeBot (código HTTP não
foi 200).");
    }
} else {
    Serial.print("Erro na conexão HTTP ou envio: ");
    Serial.println(http.errorToString(httpResponseCode).c_str());
}
http.end();
if (callback) {
    callback(success, httpResponseCode);
}
}

void handleMessageSent(bool success, int httpCode) {
    Serial.print("Callback executado: Envio ");
    Serial.print(success ? "bem-sucedido" : "falhou");
    Serial.print(", HTTP Code: ");
    Serial.println(httpCode);
    addAlertToHistory(success, httpCode);
}

void setup() {
    Serial.begin(115200);

    pinMode(sensorPin, INPUT);
    pinMode(ledVermelho, OUTPUT);
    pinMode(ledAmarelo, OUTPUT);
    pinMode(ledVerde, OUTPUT);

    digitalWrite(ledVermelho, LOW);
    digitalWrite(ledAmarelo, LOW);
    digitalWrite(ledVerde, LOW);

    Serial.println("Sistema de Alerta de Movimento Iniciado.");
    Serial.println("Alertas incluirão um link para uma imagem hospedada.");
    if (String(alertImageUrl) == "https://i.postimg.cc/6QRWKL0K/fa-a-o-codigo-de-
uma-imagem-de-um-bandido-comun-abrindo-uma-ma-aneta-de-uma-porta.jpg")
    {
        Serial.println("!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!");
        Serial.println("!!! ATENÇÃO: Atualize a variável 'alertImageUrl' com a URL da
sua imagem !!!");
    }
}

```



```

        alertMessage += "Imagem do evento:\n";
        alertMessage += alertImageUrl; // Adiciona a URL da imagem à
mensagem

        Serial.println("--- Mensagem de Alerta ---");
        Serial.println(alertMessage);
        Serial.println("-----");

        Serial.print("Enviando mensagem para CallMeBot...");
        sendMessage(alertMessage, handleMessageSent);
        Serial.println("Alerta enviado. Aguardando próximo evento após cooldown
de " + String(alertCooldown / 1000) + "s.");
    } else {
        Serial.println("Movimento estável detectado, mas DENTRO do cooldown
de alerta. Nenhuma mensagem será reenviada.");
    }
}
}
} else {
    if (isPotentialMotion) {
        Serial.println("Sensor LOW. Fim da sequência de movimento atual.");
        isPotentialMotion = false;
        ongoingValidatedMotion = false;
    }
}
delay(50);
}

```

3. Relatório de Desempenho e Testes do Sistema de Alerta de Movimento

3.1 Análise de Latência

A latência total do sistema é o tempo decorrido desde a detecção de um movimento até o recebimento da notificação no WhatsApp. Ela pode ser dividida em três componentes principais:

a) Latência de Detecção:

- Lógica de Detecção: O código não reage ao primeiro sinal HIGH do sensor. Ele implementa um limiar de estabilidade (MOTION_STABILITY_THRESHOLD_MS = 200), o que significa que o sensor precisa permanecer em estado alto por 200 milissegundos para que o movimento seja validado.

- Atraso do Loop: A função `loop()` tem um `delay(50)` no final. No pior caso, o sistema pode levar até 50ms para ler o estado do sensor.
- Latência Mínima de Detecção: A latência introduzida pelo próprio código antes de iniciar o processo de alerta é de aproximadamente 250ms (200ms do limiar + 50ms do loop).

b) Latência de Rede e Comunicação:

- Conexão Wi-Fi (Inicialização): O tempo para conectar ao Wi-Fi (`WiFi.begin()`) ocorre apenas na inicialização e pode variar de 2 a 10 segundos, dependendo da qualidade do sinal e da rede. Durante a operação normal, essa latência não se aplica a cada alerta.
- Requisição HTTP (CallMeBot): Esta é a fonte de latência mais significativa e variável. Envolve:
 1. Resolução DNS: Encontrar o endereço IP de `api.callmebot.com`.
 2. Estabelecimento da Conexão: O handshake TCP/IP com o servidor.
 3. Envio e Resposta: O tempo para enviar a requisição GET e receber a resposta do servidor.
- Estimativa: A latência da API pode variar de 500ms a mais de 3 segundos, dependendo da carga do servidor do CallMeBot e da qualidade da conexão com a internet.

c) Latência Total Estimada:

Somando os componentes, a latência total esperada para cada alerta é:

- Latência Total $\approx$ Latência de Detecção + Latência da API
- Latência Total $\approx 250\text{ms} + (500\text{ms a } 3000\text{ms})$
- Resultado: Entre 0.75 segundos e 3.25 segundos por alerta.

Conclusão (Latência): O sistema tem uma latência aceitável para um alerta de segurança não crítico. A maior parte do atraso vem da comunicação com a API externa, que está fora do controle do dispositivo.

3.2. Análise de Consumo de Energia

O consumo de energia é crucial para dispositivos que podem operar com baterias ou em locais remotos.

a) Perfil de Consumo do Código:

- Modo de Operação: O ESP32 está operando continuamente em Modo Ativo. A função loop() está sempre em execução, verificando o sensor e mantendo a conexão Wi-Fi ativa.
- Consumo Elevado: O Modo Ativo com Wi-Fi ligado é o modo que mais consome energia. Um ESP32 nesse estado pode consumir em média entre 80mA e 150mA.
- Componentes Adicionais:
 - Sensor PIR: Consumo muito baixo, geralmente na ordem de microampères (μ A).
 - LEDs: Consomem energia apenas quando piscam, sendo um consumo desprezível.

b) Ineficiência do Modelo Atual:

O dispositivo permanece em alto consumo mesmo quando não há movimento. Para uma aplicação de segurança que passa 99% do tempo inativa, isso é extremamente ineficiente e torna o uso de baterias inviável por longos períodos.

c) Recomendações para Otimização:

- Utilizar Deep Sleep (Sono Profundo): A otimização mais impactante seria redesenhar o código para usar o modo *Deep Sleep* do ESP32.
 1. Funcionamento: O ESP32 entraria em um estado de consumo ultrabaixo (tipicamente 10-20 μ A).
 2. Acionamento (Wake-up): O pino do sensor PIR seria configurado como uma fonte de "despertar" (interruptor externo). Quando o sensor detectasse movimento, ele enviaria um sinal que "acordaria" o ESP32.
 3. Execução: Ao acordar, o ESP32 se conectaria ao Wi-Fi, enviaria o alerta e voltaria a dormir.
- Impacto: A implementação do *Deep Sleep* poderia aumentar a autonomia de uma bateria de dias para meses, transformando-o em um dispositivo verdadeiramente sem fio e de longa duração.

Conclusão (Energia): O design atual é simples, mas altamente ineficiente em termos de energia. A migração para um modelo baseado em *Deep Sleep* é fortemente recomendada.

3.3 Análise da Eficiência da Comunicação

Esta análise foca na robustez e na otimização da troca de dados com a API.

a) Protocolo e Payload:

- Protocolo: O uso de HTTP GET é simples e eficaz para esta aplicação. Embora protocolos como MQTT possam ser mais leves, o HTTP é universal e bem suportado.
- Payload: A carga de dados é mínima (número de telefone, chave de API e a mensagem de texto com uma URL), o que é muito eficiente. A função `urlencode()` é usada corretamente para garantir que a mensagem seja transmitida sem erros.

b) Tratamento de Erros e Robustez:

- Verificação de Resposta: O código verifica o código de resposta HTTP (`httpResponseCode`). Ele trata o sucesso (código 200) de forma diferente de outros códigos, o que é uma boa prática. A função `http.errorToString()` também ajuda a diagnosticar falhas de conexão.
- Cooldown de Alertas (`alertCooldown`): A implementação de um cooldown de 5 segundos (5000ms) é excelente. Isso previne o envio de múltiplas mensagens para o mesmo evento de movimento contínuo, evitando:
 - Spam para o usuário.
 - Bans temporários ou sobrecarga da API CallMeBot.
 - Consumo desnecessário de dados e energia.
- Histórico de Alertas (`alertHistory`): A criação de um histórico dos últimos 10 eventos é uma ótima ferramenta para *debugging*. Ela permite verificar se as falhas estão relacionadas à API (analisando os códigos de erro) ou a outros fatores, sem a necessidade de um monitor serial conectado o tempo todo.

Conclusão (Comunicação): A estratégia de comunicação é robusta e bem implementada. O tratamento de erros, o cooldown e o histórico de eventos demonstram um design cuidadoso e eficiente para a tarefa proposta.

4 Conclusão

A evolução deste código ESP32 ilustra bem o processo de desenvolvimento iterativo. Partido de um sistema funcional, mas básico, e, através de sucessivas solicitações e implementações, buscando otimizar o desempenho, refinar a lógica e adaptar a forma de comunicação.

A transição de um link para câmera IP, para um ícone textual, e depois para um link de uma imagem específica hospedada, demonstra a exploração de diferentes maneiras de fornecer feedback visual dentro das restrições das ferramentas.

REFERÊNCIAS

ENGEASIER | AUTOMAÇÃO COM ARDUINO E ESP32. **RECEBENDO ALERTAS NO WHATSAPP DA AUTOMAÇÃO COM ESP32**. Disponível em: <<https://www.youtube.com/watch?v=Hmu7P1YRIOM>>. Acesso em: 20 abr. 2025.

QSP TECNOLOGIA. **Alarme Residencial com ESP32 e WhatsApp**. Disponível em: <<https://www.youtube.com/watch?v=-PU-xsEQoJw>>. Acesso em: 20 abr. 2025.

Free API to Send Whatsapp Messages. Disponível em: <<https://www.callmebot.com/blog/free-api-whatsapp-messages/>>. Acesso em: 1 jun. 2025.

Postimages — free image hosting / image upload. Disponível em: <<https://postimages.org/>>. Acesso em: 15 jun. 2025.